Assignment 4

MIPS Game Project

Module 4: MIPS Assembly, Part 2

Student	Ethan Brothers
Programs	TicTacToe.asm BlackJack.asm CustomGame.asm
Environment	MARS MIPS Simulator - console and graphical tools
Repository	CS240 / Assignment 4 MIPS Game Project

This report contains the three required MIPS games, their design notes, run procedures, testing plan, source code, and source acknowledgments.

Verification note: The PDF documents the expected test cases. Add screenshots and observed results from the MARS runs before final submission.

1. Project Summary and Run Procedure

This project implements three interactive MIPS games for Module 4. The games demonstrate reusable procedures, stack conventions, arrays, branching, loops, system calls, user input, graphical output, and game-state management.

Games included: Tic-Tac-Toe, Blackjack, and Snake as the custom game.

Simulator: MARS MIPS simulator.

Course reference: ZyBooks Chapter 4.

The uploaded source files are included in the final section of this document. In a GitHub repository, they can be renamed to `TicTacToe.asm`, `BlackJack.asm`, and `CustomGame.asm`.

2. Design Notes

Tic-Tac-Toe

Tic-Tac-Toe stores the board as nine words in memory. A value of 1 represents Player X, a value of 2 represents Player O, and zero represents an empty square. The player selects squares through the console while the board and pieces are drawn in the MARS Bitmap Display. After each valid move, the program checks all eight possible winning lines and then checks for a draw.

Blackjack

Blackjack stores the player and dealer hands in separate byte arrays. Cards are generated using MARS random-number system calls. The hand-value procedure handles face cards and converts Aces from 11 to 1 when needed to prevent a bust. The player chooses whether to hit or stand, and the dealer draws until reaching at least 17.

Snake

Snake uses the MARS Keyboard and Display MMIO Simulator. The snake body, walls, and food are stored directly in display memory. Each body cell stores direction information in an unused part of the display word, allowing the tail to follow the body without a separate large body array. The game checks for food, wall collisions, and self-collisions.

3. Procedures and Game State

Game	Important procedures	Game state
Tic-Tac-Toe	initialize_game, clear_screen, draw_grid, draw_piece, check_winner, check_line	Nine-word board array; player and move count registers
Blackjack	seed_random, random_card, deal_card_safe, hand_value, player_turn, dealer_turn, decide_winner	Player and dealer byte arrays; hand counts in registers
Snake	initialize_game, draw_walls, place_food, read_input, move_snake, show_game_over	Display memory stores walls, food, and the snake body

4. Running Instructions

General MARS steps

1. Open the desired `.asm` file in MARS. 2. Open the required MARS tool, if applicable. 3. Assemble the program. 4. Run the program. 5. Enter input in the MARS console or simulator window.

Tic-Tac-Toe Bitmap Display settings

Blackjack controls

Enter 1 to hit, 2 to stand, 1 to play another round, or 0 to quit. Invalid hit, stand, and replay options are rejected.

Snake MMIO settings

Open Tools -> Keyboard and Display MMIO Simulator and connect it to MARS. Click in the simulator input area. Use W or w for up, A or a for left, S or s for down, and D or d for right. The game ends when the snake hits a wall or itself.

5. Testing Plan

The following tests should be run in MARS and supported with screenshots in the final submission. The table records the expected behavior; actual input and observed output should be added after running each test.

Test	Input/action	Expected result	Observed result
T1 - Tic-Tac-Toe win	Complete a row, column, or diagonal	Correct player-win message and graphical board	Run in MARS
T2 - Blackjack round	Hit and then stand	Dealer completes its turn and winner is reported	Run in MARS
T3 - Snake food	Move to blue food	Snake grows and new food is placed	Run in MARS
E1 - Tic-Tac-Toe invalid move	Enter 0, 10, or an occupied square	Invalid-move message; turn remains active	Run in MARS
E2 - Blackjack invalid input	Enter an option other than 1 or 2	Invalid-choice message and prompt repeats	Run in MARS
E3 - Snake invalid key	Press a key other than W/A/S/D	Key is ignored and game continues	Run in MARS
E4 - Snake collision	Move into a wall or body	Game-over message is displayed	Run in MARS

Evidence note: this draft does not invent screenshots or observed test results. Add the actual MARS screenshots and replace the “Run in MARS” entries after testing.

6. Assignment Checklist

The project includes reusable procedures, stack use, arrays or memory-based state, branching, loops, system calls, user input, graphical output in the graphical games, game logic, and basic invalid-input or collision handling. Before submission, verify the Blackjack graphic-display requirement and include screenshots for all required evidence.

7. Sources and Acknowledgments

This project was created for the Module 4 MIPS Game Project using concepts from ZyBooks Chapter 4. The MARS MIPS simulator and its Bitmap Display and Keyboard and Display MMIO Simulator tools were used for execution and testing. Any outside examples or assistance should be acknowledged according to the course requirements.

8. MIPS Source Code

The complete source code is included below. The source is reproduced from the uploaded project files.

Tic-Tac-Toe: TicTacToe.asm

```
# Phase 1: Graphical Tic-Tac-Toe for MARS
# Bitmap Display settings:
#   Unit width: 8, Unit height: 8
#   Display width: 256, Display height: 256
#   Base address: 0x10008000
# Instructions/input appear in the MARS console; the board appears graphically.

.eqv DISPLAY_BASE 0x10008000

.data
displayAddress: .word 0x10008000
board:        .space 36
title:        .asciiz "\n=== TIC-TAC-TOE ===\n"
instructions: .asciiz "Choose a square from 1 to 9:\n\n 1 | 2 | 3\n---+---+---\n 4 | 5 | 6\n---+---+---\n 7 | 8 | 9\n\n"
prompt_x:     .asciiz "Player X, choose a square: "
prompt_o:     .asciiz "Player O, choose a square: "
invalid:      .asciiz "Invalid choice. Enter an empty square from 1 to 9.\n"
x_wins:       .asciiz "Player X wins!\n"
o_wins:       .asciiz "Player O wins!\n"
draw_msg:     .asciiz "The game is a draw.\n"
again_prompt: .asciiz "Play again? Enter 1 for yes or 0 for no: "
bye:          .asciiz "Thanks for playing!\n"

.text
.globl main

main:
    la   $s0, board

new_game:
    jal  initialize_game
    jal  clear_screen
    jal  draw_grid
    li   $v0, 4
    la   $a0, title
    syscall
    li   $v0, 4
    la   $a0, instructions
    syscall
    li   $s1, 1                # current player: 1 = X, 2 = O
    li   $s2, 0                # move count

game_loop:
    li   $v0, 4
    li   $t7, 1
    beq  $s1, $t7, print_x_prompt
    la   $a0, prompt_o
    syscall
    j    get_move

print_x_prompt:
    la   $a0, prompt_x
    syscall

get_move:
    li   $v0, 5
    syscall
    move $t0, $v0
    slti $t7, $t0, 1
    bne  $t7, $zero, bad_move
    slti $t7, $t0, 10
    beq  $t7, $zero, bad_move
    addi $t1, $t0, -1
    sll  $t2, $t1, 2
    addu $t3, $s0, $t2
    lw   $t4, 0($t3)
    bne  $t4, $zero, bad_move

    sw   $s1, 0($t3)
    addi $s2, $s2, 1

    li   $t5, 3
    div  $t1, $t5
    mflo $a1                   # row
    mfhi $a0                   # column
    move $a2, $s1
    jal  draw_piece

    jal  check_winner
    move $s3, $v0
```

```
    bne  $s3, $zero, game_over
    li   $t6, 9
    beq  $s2, $t6, game_draw

    li   $t6, 3
    sub  $s1, $t6, $s1
    j    game_loop

bad_move:
    li   $v0, 4
    la   $a0, invalid
    syscall
    j    game_loop

game_over:
    li   $v0, 4
    li   $t7, 1
    beq  $s3, $t7, show_x_win
    la   $a0, o_wins
    syscall
    j    ask_again
show_x_win:
    la   $a0, x_wins
    syscall
    j    ask_again

game_draw:
    li   $v0, 4
    la   $a0, draw_msg
    syscall

ask_again:
    li   $v0, 4
    la   $a0, again_prompt
    syscall
    li   $v0, 5
    syscall
    li   $t7, 1
    beq  $v0, $t7, new_game
    li   $v0, 4
    la   $a0, bye
    syscall
    li   $v0, 10
    syscall

# Clear the nine board words.
initialize_game:
    move $t0, $s0
    li   $t1, 0
init_loop:
    sw   $zero, 0($t0)
    addi $t0, $t0, 4
    addi $t1, $t1, 1
    li   $t7, 9
    blt  $t1, $t7, init_loop
    jr   $ra

# Fill the 32x32 Bitmap Display with dark gray.
clear_screen:
    addi $sp, $sp, -4
    sw   $ra, 0($sp)
    li   $s4, 0                 # y
clear_y:
    li   $s5, 0                 # x
clear_x:
    move $a0, $s5
    move $a1, $s4
    li   $a2, 0x00202020
    jal  draw_pixel
    addi $s5, $s5, 1
    li   $t7, 32
    blt  $s5, $t7, clear_x
    addi $s4, $s4, 1
    li   $t7, 32
    blt  $s4, $t7, clear_y
    lw   $ra, 0($sp)
    addi $sp, $sp, 4
    jr   $ra

# Draw vertical lines at x=8,16 and horizontal lines at y=8,16.
draw_grid:
    addi $sp, $sp, -4
    sw   $ra, 0($sp)

    # The board is 24 units wide and high.
    # Each cell is 8 units, so boundaries are at 0, 8, 16, and 24.
    li   $s4, 0
vertical_lines:
    li   $a0, 0
```

```
    move $a1, $s4
    li   $a2, 0x00FFFFFF
    jal  draw_pixel

    li   $a0, 8
    move $a1, $s4
    li   $a2, 0x00FFFFFF
    jal  draw_pixel

    li   $a0, 16
    move $a1, $s4
    li   $a2, 0x00FFFFFF
    jal  draw_pixel

    li   $a0, 24
    move $a1, $s4
    li   $a2, 0x00FFFFFF
    jal  draw_pixel

    addi $s4, $s4, 1
    li   $t7, 25
    blt  $s4, $t7, vertical_lines

    li   $s4, 0
horizontal_lines:
    move $a0, $s4
    li   $a1, 0
    li   $a2, 0x00FFFFFF
    jal  draw_pixel

    move $a0, $s4
    li   $a1, 8
    li   $a2, 0x00FFFFFF
    jal  draw_pixel

    move $a0, $s4
    li   $a1, 16
    li   $a2, 0x00FFFFFF
    jal  draw_pixel

    move $a0, $s4
    li   $a1, 24
    li   $a2, 0x00FFFFFF
    jal  draw_pixel

    addi $s4, $s4, 1
    li   $t7, 25
    blt  $s4, $t7, horizontal_lines

    lw   $ra, 0($sp)
    addi $sp, $sp, 4
    jr   $ra

# a0 = column, a1 = row, a2 = player.
draw_piece:
    li   $t7, 1
    beq  $a2, $t7, draw_x
    j    draw_o

draw_x:
    addi $sp, $sp, -4
    sw   $ra, 0($sp)
    sll  $s4, $a0, 3
    sll  $s5, $a1, 3
    addi $s4, $s4, 2
    addi $s5, $s5, 2
    li   $s6, 0
draw_x_loop:
    addu $a0, $s4, $s6
    addu $a1, $s5, $s6
    li   $a2, 0x00FF4040
    jal  draw_pixel
    addu $a0, $s4, $s6
    sub  $a1, $s5, $s6
    addi $a1, $a1, 3
    li   $a2, 0x00FF4040
    jal  draw_pixel
    addi $s6, $s6, 1
    li   $t7, 4
    blt  $s6, $t7, draw_x_loop
    lw   $ra, 0($sp)
    addi $sp, $sp, 4
    jr   $ra

draw_o:
    addi $sp, $sp, -4
    sw   $ra, 0($sp)
    sll  $s4, $a0, 3
    sll  $s5, $a1, 3
```

```
    addi $s4, $s4, 2
    addi $s5, $s5, 2
    li   $s6, 0
draw_o_loop:
    addu $a0, $s4, $s6
    move $a1, $s5
    li   $a2, 0x004080FF
    jal  draw_pixel
    addu $a0, $s4, $s6
    addi $a1, $s5, 3
    li   $a2, 0x004080FF
    jal  draw_pixel
    move $a0, $s4
    addu $a1, $s5, $s6
    li   $a2, 0x004080FF
    jal  draw_pixel
    addi $a0, $s4, 3
    addu $a1, $s5, $s6
    li   $a2, 0x004080FF
    jal  draw_pixel
    addi $s6, $s6, 1
    li   $t7, 4
    blt  $s6, $t7, draw_o_loop
    lw   $ra, 0($sp)
    addi $sp, $sp, 4
    jr   $ra

# a0=x unit, a1=y unit, a2=0x00RRGGBB color.
draw_pixel:
    sll  $t0, $a1, 5
    addu $t0, $t0, $a0
    sll  $t0, $t0, 2
    lw   $t1, displayAddress
    addu $t1, $t1, $t0
    sw   $a2, 0($t1)
    jr   $ra

# Returns v0=0 for no winner, v0=1 for X, v0=2 for O.
check_winner:
    addi $sp, $sp, -4
    sw   $ra, 0($sp)
    li $a0, 0
    li $a1, 1
    li $a2, 2
    jal check_line
    bne $v0,$zero,winner_done
    li $a0, 3
    li $a1, 4
    li $a2, 5
    jal check_line
    bne $v0,$zero,winner_done
    li $a0, 6
    li $a1, 7
    li $a2, 8
    jal check_line
    bne $v0,$zero,winner_done
    li $a0, 0
    li $a1, 3
    li $a2, 6
    jal check_line
    bne $v0,$zero,winner_done
    li $a0, 1
    li $a1, 4
    li $a2, 7
    jal check_line
    bne $v0,$zero,winner_done
    li $a0, 2
    li $a1, 5
    li $a2, 8
    jal check_line
    bne $v0,$zero,winner_done
    li $a0, 0
    li $a1, 4
    li $a2, 8
    jal check_line
    bne $v0,$zero,winner_done
    li $a0, 2
    li $a1, 4
    li $a2, 6
    jal check_line
winner_done:
    lw   $ra, 0($sp)
    addi $sp, $sp, 4
    jr   $ra

# a0,a1,a2 are board indices for one line.
check_line:
    sll  $t0, $a0, 2
    addu $t0, $s0, $t0
```

```
      lw   $t3, 0($t0)
      beq  $t3, $zero, no_line
      sll  $t0, $a1, 2
      addu $t0, $s0, $t0
      lw   $t4, 0($t0)
      bne  $t3, $t4, no_line
      sll  $t0, $a2, 2
      addu $t0, $s0, $t0
      lw   $t5, 0($t0)
      bne  $t3, $t5, no_line
      move $v0, $t3
      jr   $ra
  no_line:
      move $v0, $zero
      jr   $ra
```

Blackjack: BlackJack.asm

```
# Phase 2: Blackjack for MARS
# Console game with procedures, arrays, loops, branching, input validation,
# dealer/player turn logic, Ace handling, bust detection, capacity checks, and replay.

.eqv MAX_HAND 12

.data
player_hand: .space 12
dealer_hand: .space 12

title:        .asciiz "\n=== BLACKJACK ===\n"
rules:        .asciiz "Get as close to 21 as possible without going over.\n"
player_text:  .asciiz "Your hand: "
dealer_text:  .asciiz "Dealer's hand: "
hidden_text:  .asciiz "[hidden]"
space_text:   .asciiz " "
value_text:   .asciiz "  Value: "
hit_prompt:   .asciiz "\nEnter 1 to hit or 2 to stand: "
invalid:      .asciiz "Invalid choice. Enter 1 or 2.\n"
replay_invalid:.asciiz "Invalid choice. Enter 1 to play again or 0 to quit.\n"
bust_player:  .asciiz "\nYou busted. Dealer wins.\n"
bust_dealer:  .asciiz "\nDealer busted. You win!\n"
player_wins:  .asciiz "\nYou win!\n"
dealer_wins:  .asciiz "\nDealer wins.\n"
tie_text:     .asciiz "\nIt's a tie.\n"
hand_limit:   .asciiz "The hand is full, so you must stand.\n"
replay_text:  .asciiz "\nPlay again? Enter 1 for yes or 0 for no: "
bye:          .asciiz "Thanks for playing Blackjack!\n"

ace_text:     .asciiz "A"
two_text:     .asciiz "2"
three_text:   .asciiz "3"
four_text:    .asciiz "4"
five_text:    .asciiz "5"
six_text:     .asciiz "6"
seven_text:   .asciiz "7"
eight_text:   .asciiz "8"
nine_text:    .asciiz "9"
ten_text:     .asciiz "10"
jack_text:    .asciiz "J"
queen_text:   .asciiz "Q"
king_text:    .asciiz "K"

.text
.globl main

main:
    jal  seed_random

new_round:
    la   $s3, player_hand
    la   $s4, dealer_hand
    li   $s1, 0                 # player count
    li   $s2, 0                 # dealer count

    # Two player cards.
    move $a0, $s3
    move $a1, $s1
    jal  deal_card_safe
    addi $s1, $s1, 1
    move $a0, $s3
    move $a1, $s1
    jal  deal_card_safe
    addi $s1, $s1, 1

    # Two dealer cards.
    move $a0, $s4
    move $a1, $s2
    jal  deal_card_safe
    addi $s2, $s2, 1
    move $a0, $s4
    move $a1, $s2
    jal  deal_card_safe
    addi $s2, $s2, 1

    li   $v0, 4
    la   $a0, title
    syscall
    li   $v0, 4
    la   $a0, rules
    syscall

    # Show dealer's first card, hiding the second.
    li   $v0, 4
    la   $a0, dealer_text
    syscall
    lbu  $a0, 0($s4)
```

```
    jal  print_card
    li   $v0, 4
    la   $a0, space_text
    syscall
    li   $v0, 4
    la   $a0, hidden_text
    syscall
    li   $v0, 11
    li   $a0, 10
    syscall

    jal  print_player_hand
    jal  player_turn
    li   $t7, 1
    beq  $v0, $t7, player_busted

    jal  dealer_turn
    jal  print_dealer_hand
    jal  decide_winner
    j    replay

player_busted:
    li   $v0, 4
    la   $a0, bust_player
    syscall
    j    replay

# Seed random generator 0 using the system time.
seed_random:
    li   $v0, 30
    syscall
    move $t0, $a0
    li   $v0, 40
    li   $a0, 0
    move $a1, $t0
    syscall
    jr   $ra

# Returns a card from 1 through 13 in v0.
random_card:
    li   $v0, 42
    li   $a0, 0
    li   $a1, 13
    syscall
    addi $v0, $a0, 1
    jr   $ra

# a0=array base, a1=index, a2=card.
store_card:
    addu $t0, $a0, $a1
    sb   $a2, 0($t0)
    jr   $ra

# a0=array base, a1=count; returns total in v0.
hand_value:
    li   $t0, 0                 # total
    li   $t1, 0                 # aces
    li   $t2, 0                 # index
value_loop:
    beq  $t2, $a1, adjust_aces
    addu $t3, $a0, $t2
    lbu  $t4, 0($t3)
    li   $t7, 1
    beq  $t4, $t7, add_ace
    li   $t7, 10
    ble  $t4, $t7, add_number
    addi $t0, $t0, 10
    addi $t2, $t2, 1
    j    value_loop
add_ace:
    addi $t0, $t0, 11
    addi $t1, $t1, 1
    addi $t2, $t2, 1
    j    value_loop
add_number:
    addu $t0, $t0, $t4
    addi $t2, $t2, 1
    j    value_loop
adjust_aces:
    li   $t7, 21
ace_adjust_loop:
    ble  $t0, $t7, value_done
    beq  $t1, $zero, value_done
    addi $t0, $t0, -10
    addi $t1, $t1, -1
    j    ace_adjust_loop
value_done:
    move $v0, $t0
    jr   $ra
```

```
# a0=card value; prints card name.
print_card:
    li $t7,1
    beq $a0,$t7,card_ace
    li $t7,2
    beq $a0,$t7,card_two
    li $t7,3
    beq $a0,$t7,card_three
    li $t7,4
    beq $a0,$t7,card_four
    li $t7,5
    beq $a0,$t7,card_five
    li $t7,6
    beq $a0,$t7,card_six
    li $t7,7
    beq $a0,$t7,card_seven
    li $t7,8
    beq $a0,$t7,card_eight
    li $t7,9
    beq $a0,$t7,card_nine
    li $t7,10
    beq $a0,$t7,card_ten
    li $t7,11
    beq $a0,$t7,card_jack
    li $t7,12
    beq $a0,$t7,card_queen
    la $a0, king_text
    j print_card_text
card_ace:
    la $a0, ace_text
    j print_card_text
card_two:
    la $a0, two_text
    j print_card_text
card_three:
    la $a0, three_text
    j print_card_text
card_four:
    la $a0, four_text
    j print_card_text
card_five:
    la $a0, five_text
    j print_card_text
card_six:
    la $a0, six_text
    j print_card_text
card_seven:
    la $a0, seven_text
    j print_card_text
card_eight:
    la $a0, eight_text
    j print_card_text
card_nine:
    la $a0, nine_text
    j print_card_text
card_ten:
    la $a0, ten_text
    j print_card_text
card_jack:
    la $a0, jack_text
    j print_card_text
card_queen:
    la $a0, queen_text
    j print_card_text
print_card_text:
    li $v0, 4
    syscall
    jr $ra

# Print player's complete hand and total.
print_player_hand:
    addi $sp, $sp, -4
    sw   $ra, 0($sp)
    li   $v0, 4
    la   $a0, player_text
    syscall
    li   $t0, 0
player_print_loop:
    beq  $t0, $s1, player_print_value
    addu $t1, $s3, $t0
    lbu  $a0, 0($t1)
    jal  print_card
    li   $v0, 4
    la   $a0, space_text
    syscall
    addi $t0, $t0, 1
    j    player_print_loop
player_print_value:
    li   $v0, 4
```

```
    la   $a0, value_text
    syscall
    move $a0, $s3
    move $a1, $s1
    jal  hand_value
    move $a0, $v0
    li   $v0, 1
    syscall
    li   $v0, 11
    li   $a0, 10
    syscall
    lw   $ra, 0($sp)
    addi $sp, $sp, 4
    jr   $ra

# Player turn. Returns v0=0 for stand, v0=1 for bust.
player_turn:
    addi $sp, $sp, -4
    sw   $ra, 0($sp)
player_choice:
    li   $v0, 4
    la   $a0, hit_prompt
    syscall
    li   $v0, 5
    syscall
    li   $t7, 1
    beq  $v0, $t7, player_hit
    li   $t7, 2
    beq  $v0, $t7, player_stand
    li   $v0, 4
    la   $a0, invalid
    syscall
    j    player_choice

player_hit:
    # Prevent writing past the 12-byte player_hand array.
    li   $t7, MAX_HAND
    bge  $s1, $t7, hand_full_stand
    move $a0, $s3
    move $a1, $s1
    jal  deal_card_safe
    addi $s1, $s1, 1
    jal  print_player_hand
    move $a0, $s3
    move $a1, $s1
    jal  hand_value
    li   $t7, 21
    bgt  $v0, $t7, player_bust_return
    j    player_choice

hand_full_stand:
    li   $v0, 4
    la   $a0, hand_limit
    syscall
    j    player_stand

player_bust_return:
    li   $v0, 1
    lw   $ra, 0($sp)
    addi $sp, $sp, 4
    jr   $ra
player_stand:
    move $v0, $zero
    lw   $ra, 0($sp)
    addi $sp, $sp, 4
    jr   $ra

# Dealer hits until total is 17 or more.
dealer_turn:
    addi $sp, $sp, -4
    sw   $ra, 0($sp)
dealer_loop:
    move $a0, $s4
    move $a1, $s2
    jal  hand_value
    li   $t7, 17
    bge  $v0, $t7, dealer_done
    # Stop safely if the dealer reaches the array capacity.
    li   $t7, MAX_HAND
    bge  $s2, $t7, dealer_done
    move $a0, $s4
    move $a1, $s2
    jal  deal_card_safe
    addi $s2, $s2, 1
    j    dealer_loop
dealer_done:
    lw   $ra, 0($sp)
    addi $sp, $sp, 4
    jr   $ra
```

```
# Print dealer's complete hand and total.
print_dealer_hand:
    addi $sp, $sp, -4
    sw   $ra, 0($sp)
    li   $v0, 4
    la   $a0, dealer_text
    syscall
    li   $t0, 0
dealer_print_loop:
    beq  $t0, $s2, dealer_print_value
    addu $t1, $s4, $t0
    lbu  $a0, 0($t1)
    jal  print_card
    li   $v0, 4
    la   $a0, space_text
    syscall
    addi $t0, $t0, 1
    j    dealer_print_loop
dealer_print_value:
    li   $v0, 4
    la   $a0, value_text
    syscall
    move $a0, $s4
    move $a1, $s2
    jal  hand_value
    move $a0, $v0
    li   $v0, 1
    syscall
    li   $v0, 11
    li   $a0, 10
    syscall
    lw   $ra, 0($sp)
    addi $sp, $sp, 4
    jr   $ra

# Compare totals and print the result.
decide_winner:
    addi $sp, $sp, -4
    sw   $ra, 0($sp)
    move $a0, $s3
    move $a1, $s1
    jal  hand_value
    move $t0, $v0
    move $a0, $s4
    move $a1, $s2
    jal  hand_value
    move $t1, $v0
    li   $t7, 21
    bgt  $t1, $t7, dealer_bust_result
    bgt  $t0, $t1, player_win_result
    blt  $t0, $t1, dealer_win_result
    li   $v0, 4
    la   $a0, tie_text
    syscall
    j    winner_return
dealer_bust_result:
    li   $v0, 4
    la   $a0, bust_dealer
    syscall
    j    winner_return
player_win_result:
    li   $v0, 4
    la   $a0, player_wins
    syscall
    j    winner_return
dealer_win_result:
    li   $v0, 4
    la   $a0, dealer_wins
    syscall
winner_return:
    lw   $ra, 0($sp)
    addi $sp, $sp, 4
    jr   $ra

# Ask whether to start another round.
replay:
    li   $v0, 4
    la   $a0, replay_text
    syscall
    li   $v0, 5
    syscall
    li   $t7, 1
    beq  $v0, $t7, new_round
    li   $t7, 0
    beq  $v0, $t7, exit_game
    li   $v0, 4
    la   $a0, replay_invalid
    syscall
    j    replay
```

```
exit_game:
    li   $v0, 4
    la   $a0, bye
    syscall
    li   $v0, 10
    syscall

# a0=array base, a1=index; draw and store a random card.
deal_card_safe:
    addi $sp, $sp, -12
    sw   $ra, 8($sp)
    sw   $a0, 0($sp)
    sw   $a1, 4($sp)
    li   $t7, MAX_HAND
    bge  $a1, $t7, deal_failed
    jal  random_card
    move $a2, $v0
    lw   $a0, 0($sp)
    lw   $a1, 4($sp)
    jal  store_card
    li   $v0, 1
    lw   $ra, 8($sp)
    addi $sp, $sp, 12
    jr   $ra

deal_failed:
    move $v0, $zero
    lw   $ra, 8($sp)
    addi $sp, $sp, 12
    jr   $ra
```

Snake - Custom Game: CustomGame.asm

```
# Compact Snake for MARS
# Bitmap Display: unit 8x8, display 512x256, base 0x10008000 ($gp)
# Keyboard and Display MMIO Simulator: connect it, then use W/A/S/D.
# The compact body representation is preserved: each body cell stores its
# next-direction byte in the unused top byte of the display word.

.eqv DELAY 100

.data
instructions: .asciiz "\n=== SNAKE ===\nUse W/A/S/D to move. Eat blue food. Avoid red walls and yourself.\n\n"
game_over_msg:.asciiz "\nGame over! Reset and run to play again.\n"

.text
.globl main

main:
    lui   $s6, 0xFFFF             # MMIO base and red wall color
    li    $s7, 0xC00140FF         # packed directions and blue food color
    jal   initialize_game
    jal   draw_walls
    jal   place_food
    li    $v0, 4
    la    $a0, instructions
    syscall

game_loop:
    sw    $s2, 0($s5)             # paint the current head
    li    $v0, 30                # current time
    syscall
    addiu $t8, $a0, DELAY         # busy-wait so the GUI stays responsive
wait:
    li    $v0, 30
    syscall
    slt   $t0, $a0, $t8
    bnez  $t0, wait

    jal   read_input
    jal   move_snake
    li    $t7, 1
    beq   $v0, $t7, ate_food
    li    $t7, 2
    beq   $v0, $t7, end_game
    j     game_loop

ate_food:
    jal   place_food
    j     game_loop

end_game:
    jal   show_game_over
    li    $v0, 10
    syscall

# Initialize the snake and its direction. The head starts at row 16, col 32.
initialize_game:
    addiu $s5, $gp, 4224         # head position
    move  $s3, $s5               # tail position
    li    $s4, 0x01000000        # initially move right
    ori   $s2, $s4, 0xFF00       # green body/head word
    jr    $ra

# Draw red walls around the 64x32-unit play area.
draw_walls:
    addiu $t0, $gp, 256
    addiu $t1, $gp, 8192
wall_loop:
    addiu $t0, $t0, -8
    addiu $t1, $t1, -256
    sw    $s6, 0($t0)
    sw    $s6, 4($t0)
    sw    $s6, 7936($t0)
    sw    $s6, 7940($t0)
    sw    $s6, 0($t1)
    sw    $s6, 252($t1)
    bne   $t0, $gp, wall_loop
    jr    $ra

# Place blue food in an empty display cell using the current clock value.
place_food:
    li    $v0, 30
    syscall
food_probe:
    andi  $t0, $a0, 0x1FFC
    addu  $t0, $t0, $gp
    lw    $t1, 0($t0)
    addiu $a0, $a0, 156
    bnez  $t1, food_probe
```

```
    sw    $s7, 0($t0)
    jr    $ra

# Convert only valid W/A/S/D keys into a direction. Other keys are ignored.
read_input:
    lw    $t0, 4($s6)
    li    $t1, 97               # a
    beq   $t0, $t1, valid_key
    li    $t1, 65               # A
    beq   $t0, $t1, valid_key
    li    $t1, 115              # s
    beq   $t0, $t1, valid_key
    li    $t1, 83               # S
    beq   $t0, $t1, valid_key
    li    $t1, 100              # d
    beq   $t0, $t1, valid_key
    li    $t1, 68               # D
    beq   $t0, $t1, valid_key
    li    $t1, 119              # w
    beq   $t0, $t1, valid_key
    li    $t1, 87               # W
    bne   $t0, $t1, read_done
valid_key:
    sll   $t0, $t0, 2
    andi  $t0, $t0, 0x18
    srav  $t0, $s7, $t0
    sll   $t0, $t0, 24          # direction stored in the top byte
    addu  $t1, $t0, $s4
    beqz  $t1, read_done        # do not reverse into the snake
    move  $s4, $t0
read_done:
    jr    $ra

# Move one step. Returns v0=0 normal, v0=1 food, v0=2 collision.
move_snake:
    ori   $s2, $s4, 0xFF00      # old head points toward new head
    sw    $s2, 0($s5)
    sra   $t0, $s4, 22          # direction byte -> byte offset
    addu  $s5, $s5, $t0
    lw    $t0, 0($s5)
    beq   $t0, $s7, move_food
    bnez  $t0, move_collision

    lw    $t0, 0($s3)            # follow the tail's direction
    sw    $zero, 0($s3)
    sra   $t0, $t0, 22
    addu  $s3, $s3, $t0
    move  $v0, $zero
    jr    $ra
move_food:
    li    $v0, 1
    jr    $ra
move_collision:
    li    $v0, 2
    jr    $ra

show_game_over:
    li    $v0, 4
    la    $a0, game_over_msg
    syscall
    jr    $ra
```